AULA 40

Anti-Corruption Layer e contratos de API

Java

Conceitos, exemplos em Java e desafio aplicado

28/10/2026 · 14:00–17:00 · Java e Quarkus

FIAP · FORMAÇÃO EM BACKEND JAVA · TURMA 2

Roteiro do encontro: 3 horas

- 14:00–14:05 Abertura, cenário e objetivos.
- 14:05–15:00 Teoria com exemplos e discussão.
- 15:00–15:30 Prática guiada pelo professor.
- 15:30–15:45 Intervalo.
- 15:45–16:45 Desafio em duplas.
- 16:45–17:00 Correção, demonstrações e dúvidas.

JAVA

60 + 105

60 min de abertura e teoria. 105 min de prática e discussão, com 15 min de intervalo.

Objetivos de aprendizagem

- Identificar quando o modelo externo contamina o domínio.
- Construir uma camada de tradução com porta e adaptador em Java.
- Definir um contrato HTTP com erros e campos consistentes.
- Testar conversões e mudanças incompatíveis no legado.

JAVA

Entrega

Ao final, cada dupla demonstra a solução e explica uma decisão técnica.

CONCEITOS E EXEMPLOS

Anti-Corruption Layer e contratos de API

Uma decisão de arquitetura aplicada a um problema observável

O cenário da aula

- O legado identifica clientes por COD_CLI e situação A ou I.
- A nova API expõe id, nome e ativo com nomes claros.
- Consumidores não devem conhecer os códigos internos do legado.
- Uma mudança externa deve afetar um adaptador delimitado.

JAVA

Fronteira

A ACL protege a linguagem e as regras do domínio novo. Vocabulário desta aula: cliente = a entidade de negócio (Cliente, ClienteLegado); consumidor = quem chama a nossa API; fornecedor = o sistema externo que nos fornece dados (aqui, o legado).

Anti-Corruption Layer (1)

- A ACL traduz conceitos entre modelos de sistemas diferentes.
- Ela converte dados, códigos, erros e protocolos.
- O domínio depende de uma porta definida pela aplicação.
- O adaptador concentra as particularidades do sistema externo.

JAVA

Responsabilidade

A tradução deve preservar o significado de cada informação.

Anti-Corruption Layer (2)

- ACL é um padrão de fronteira em que cada contexto mantém seu modelo e sua linguagem. A ACL converte o modelo em termos locais.
- Potenciais cenários:
  - Migração em etapas, em que o novo sistema precisa chamar o legado enquanto a migração não termina;
  - Sistema externo fora de controle, por exemplo parceiro externo, SaaS, outro módulo de um monólito modular.
- A anatomia em três partes:
  - Facade: interface simples para o resto do sistema;
  - Adapter: converte a interface externa na interna;
  - Translator/Mapper: mapeia estruturas e conceitos.

CASO DE USO

Situação hipotética

Um dado sistema de cadastro antigo chama o cliente de COD_CLI e o Pix chama de CPF do pagador.

Nenhum dos dois está errado — são dois contextos. O erro é deixar o COD_CLI aparecer dentro do Pix.

DTO externo e modelo interno

- Acoplamento difuso: quando o JSON cru do fornecedor é usado direto, a estrutura dele se espalha por controllers, serviços e testes. Se o fornecedor renomear um campo ou trocar price de string para número, todos os pontos quebram.
- Seu modelo de dados não é a sua API!
- No nosso exemplo:
  - ClienteLegado representa o contrato recebido.
  - Cliente representa o vocabulário da aplicação nova.
  - Um mapper faz conversões explícitas e verificáveis.
  - O recurso HTTP expõe um DTO público estável.

JAVA

Separação

Não reutilizar automaticamente o DTO externo na API pública.

Regras de tradução

- A ACL protege significado, não somente os nomes. Por exemplo, um status = "active" externo pode significar "aceito", "pago" ou "apenas gravado". Traduzir o nome sem confirmar o significado documentado é o erro mais caro, porque não gerar um erro técnico, mas gera dado errado em produção.
- Validação e saneamento na fronteira também são questão de confiança: a ACL faz mediação de sistemas com níveis de confiança diferentes. Portanto, é importante validar e sanitizar — só entra o dado certo".
- Boas práticas:
    - Valor desconhecido deve gerar uma falha identificável;
    - Campo obrigatório ausente exige tratamento explícito;
    - Conversões não devem inventar valores de negócio.

JAVA

Exemplo

Uma “situação B” chega do legado. Se o mapper fizer default -> false, o cliente fica "inativo" e perde acesso a um produto; se fizer default -> true, um cliente bloqueado judicialmente vira ativo. Nenhum dos dois é aceitável. A única resposta correta é "situação B não identificada".

Nossas regras de tradução

- Para o nosso exemplo a ser explorado:
- A e I possuem significados documentados.
- Código desconhecido deve gerar uma falha identificável.
- Campo obrigatório ausente exige tratamento explícito.
- Conversões não devem inventar valores de negócio.

JAVA

Exemplo

Situações “A” e “I” de ClienteLegado possuem significados documentados (ativo e inativo).

Uma eventual situação informada como “X” não deve transformar silenciosamente um valor de “Cliente.ativo” em false.

Contrato de API

- O contrato inclui estrutura, semântica e erros.
- Defina também limites de tamanho, nulidade e formato dos IDs.
- Por outro lado, ACL não deve conter regras de negócio nem orquestração para "manter tradução separada de decisão".
- Os erros fazem parte do contrato tanto quanto o (JSON de) sucesso. O 404 é negócio ("não existe"); o 502 é integração ("o fornecedor respondeu, mas não como combinado"). Documentar é o que permite ao consumidor reagir diferente a cada um.
- Invertendo a lógica, a sua própria API é uma ACL para os seus consumidores. "Trate como um contrato." Não exponha entidades; traduza na borda; desenhe a resposta do ponto de vista do consumidor (identificadores que ele entende — ISBN, não o id interno do banco; compor dados de várias fontes em vez de obrigar três chamadas).

JAVA

Exemplo

GET /clientes/{id} retorna 200 quando encontra o cliente.

404 representa cliente inexistente.

502 pode representar uma resposta inválida da dependência.

Evolução compatível

- Um campo opcional adicional sem alterar os existentes tende a preservar consumidores tolerantes (tolerant readers).
- Deixar espaço para evoluir: embrulhar primitivos cedo ("preco": {"valor": 10.5} em vez de "preco": 10.5), para poder acrescentar "moeda" depois sem quebrar, agrupar campos relacionados e escolher nomes que descrevem o significado para o consumidor.
- Documente depreciação e teste os consumidores relevantes. Se você não controla os consumidores, uma mudança incompatível exigirá estratégia de versão; se controla produtor e consumidor, pode mudar os dois lados juntos.
- Cenários de quebra contratual que causam incompatibilidade: remover campo obrigatório, renomear campo (date → published), mudar tipo ou estrutura (price número → objeto {amount, currency}), mudar ordem, unidade ou significado de argumentos, estrutura do corpo da requisição.

JAVA

Semântica

Um sistema envia saldo: 1500. Reais ou centavos? Se o fornecedor "corrige" de reais para centavos sem versionar, nenhum teste técnico acusa. O cliente verá R$ 15,00 ou R$ 150.000,00. Por isso "exigem contratos distintos": /v2 ou campo novo saldoCentavos, nunca o mesmo campo com outro significado.

Falhas na fronteira

- Timeout deve ter limite e mapeamento HTTP documentado.
- Registre correlação sem expor dados pessoais.
- Separar erro de tradução, erro de negócio e erro de transporte.
- Retry depende: repetir um GET costuma ser seguro, mas repetir um POST (de cobrança) costuma não ser seguro (idempotência).
- Uma ACL não corrige uma indisponibilidade do fornecedor.
- Uma ACL é um ponto único de falha e adiciona latência. Portanto, deve ser escalada junto com o serviço de destino.
- Observabilidade é parte do desenho, não acessório: IDs de correlação e logs estruturados para diagnosticar falhas de tradução.
- Custo operacional: decida se a ACL é transitória ou permanente. Quando for transitória, registre como dívida e desative quando o último chamador migrar.

JAVA

Observação

Três incidentes diferentes para respostas do legado: (a) SIT=X ⟶ 502, alguém precisa dizer o que X significa; (b) 404 ⟶ 404, o cliente não existe, ninguém precisa ser acordado; (c) > 2 s ⟶ 503, infraestrutura.
Se gerarem 500, o plantão perde uma hora descobrindo qual é qual.

Nossos modelos de cada lado

Exemplo.java

```
public record ClienteLegado(
    String COD_CLI, String NOME, String SIT) {}

public record Cliente(
    long id, String nome, boolean ativo) {}

public interface BuscaCliente {
  Cliente buscar(long id);
}
```

- Os contratos têm formatos diferentes.
- O domínio usa nomes próprios (imutável, tipado, nomes nossos).
- Separe os tipos em seus arquivos Java.
- DTO de integração (espelha o fornecedor, nomes dele).
- Tipagem forte é parte da proteção: com long id e boolean ativo, um erro de campo vira erro de compilação, não de produção.

Mapper com validação explícita

Exemplo.java

```
public Cliente converter(ClienteLegado dto) {
  boolean ativo = switch (dto.SIT()) {
    case "A" -> true;
    case "I" -> false;
    default -> throw new IllegalArgumentException(
        "Situacao desconhecida");
  };
  return new Cliente(Long.parseLong(dto.COD_CLI()),
      dto.NOME().trim(), ativo);
}
```

- Valide null e campos vazios antes de chamar o mapper.
- Uma situação nova exige decisão de domínio.
- Capture falhas na fronteira HTTP.
- O default -> throw é a aplicação de "valores desconhecidos geram erro com uma mensagem clara".
- Onde a ACL termina: o mapper decide se ativo ou inativo, mas não decide se cliente inativo pode operar (ou não).

Adaptador com REST Client

Exemplo.java

```
@Path("/cadastro")
@RegisterRestClient(configKey="legado")
public interface LegadoClient {
  @GET @Path("/{id}")
  ClienteLegado buscar(@PathParam("id") long id);
}

// application.properties
quarkus.rest-client.legado.url=http://localhost:4001
quarkus.rest-client.legado.connect-timeout=1000
quarkus.rest-client.legado.read-timeout=2000
```

- Extensões: rest-jackson e rest-client-jackson.
- O adaptador injeta @RestClient LegadoClient.
- Traduza a resposta antes de devolver Cliente.

Prática guiada: preparação

- Crie o projeto com rest-jackson e rest-client-jackson.
- Comece com um fornecedor simulado que retorna A e I.
- Implemente o mapper e a porta BuscaCliente.
- Exponha GET /clientes/{id} usando o modelo interno.

JAVA

30 minutos

O mock pode ser um bean local antes de conectar por HTTP.

Teste unitário da tradução

Exemplo.java

```
@Test
void converteClienteAtivo() {
  var entrada = new ClienteLegado("42", " Ana ", "A");
  var saida = new ClienteMapper().converter(entrada);
  assertEquals(42L, saida.id());
  assertEquals("Ana", saida.nome());
  assertTrue(saida.ativo());
}
```

- Use JUnit Jupiter.
- Teste também I e código desconhecido.
- Testes do mapper não precisam iniciar o Quarkus.

Roteiro da demonstração

- Consulte um cliente ativo pelo endpoint público.
- Mostre que COD_CLI e SIT não aparecem no JSON.
- Troque o fornecedor por uma resposta SIT=X.
- Apresente um erro controlado com código e correlação.

JAVA

Contrato

O consumidor recebe linguagem estável mesmo com modelos externos distintos.

Desafio: API protegida do legado

- Traduza A para ativo e I para inativo.
- Rejeite código inválido, nome vazio e ID não numérico.
- Devolva 404 quando o fornecedor indicar ausência.
- Mapeie payload inválido da dependência para 502.

JAVA

60 minutos

Documentem GET /clientes/{id}, respostas e exemplos.

Desafio: critérios de entrega

- Entregue os modelos, o mapper e o adaptador separados.
- Inclua pelo menos cinco testes de tradução e erro.
- O JSON público não pode conter SIT nem COD_CLI.
- Demonstre uma mudança no legado sem alterar o endpoint público.

JAVA

Aceite

Um teste deve falhar antes da correção do código desconhecido.

Extensão do desafio

- Adicione um segundo fornecedor com campos diferentes.
- Faça os dois implementarem a mesma interface BuscaCliente.
- Proponha testes de contrato para cada fornecedor.
- Documente a política para campo novo e campo removido.

JAVA

Avançado

Evite condicionar o domínio ao nome de cada fornecedor.

Correção: o adaptador traduz

Exemplo.java

```
@ApplicationScoped
public class ClienteAdapter implements BuscaCliente {
  @Inject @RestClient LegadoClient client;
  @Inject ClienteMapper mapper;

  public Cliente buscar(long id) {
    return mapper.converter(client.buscar(id));
  }
}
```

- O mapper também é um bean @ApplicationScoped.
- ExceptionMappers completam a tradução de falhas.
- O recurso HTTP injeta BuscaCliente.

Discussão e fechamento

- Qual regra pertence ao domínio e qual pertence ao mapper?
- Como a equipe descobriria um novo código de situação?
- Quais mudanças no contrato exigem versão ou depreciação?
- A ACL limita a propagação de mudanças do legado.

JAVA

Aprendizagem

Porta, adaptador e tradução explícita preservam a linguagem do domínio.

Referências e preparação

- https://quarkus.io/guides/rest-client
- https://quarkus.io/guides/openapi-swaggerui
- https://learn.microsoft.com/en-us/azure/architecture/patterns/anti-corruption-layer

JAVA

Professor

Prepare as dependências antes da aula e mantenha as versões usadas no README.